AIML ZG546 - Software Engineering for Machine Learning

Assignment II

BrakeGuard AI - Implementation, Code Quality and Testing of an ML-enabled Predictive-Maintenance System

Group No: 14

Sl. No	BITS ID	Name	Contribution of team member (Qualitative)	Percentage Contribution (out of 100)
1	2025ae05802	SAI KRISHNA VIJAYA RAM K	Refactoring into the brakeguard package (data, features, models, pipeline); research-vs-production comparison (Tasks 1, 2)	100%
2	2025ae05999	S. S SRINIVAS RAO	FastAPI REST API: schemas, status codes, error envelope, API-key security, dashboard (Task 5)	100%
3	2025ae05192	ABHISHEK ARORA	Test strategy and pytest suites: unit, data-quality, ML training/inference, integration; coverage (Tasks 6, 7)	100%
4	2025ae05200	RAJESH. R. SHENOI	Logging and error handling, black/isort/flake8/pylint, quality metrics and drift, production testing and security, report (Tasks 3, 4, 8, 9)	100%

Code: `BrakeGuard_AI_Assignment_Application/brakeguard_ai-3` (builds on the Assignment I prototype in `brakeguard_ai-2`). Implementation notebook: `notebooks/Group14.ipynb`.

Contents

1. Assignment I recap - requirements and architecture baseline

2. System overview - Assignment I architecture and its Assignment II implementation

Objective 1 - Implementation and code sharing [5 marks]

Task 1 - OOP / functional design of the ML application

Task 2 - Research code vs production code

Task 3 - Error handling and logging

Task 4 - Code formatting and linting (before / after)

Task 5 - REST API (FastAPI)

Objective 2 - Quality assurance [5 marks]

Task 6 - Types of tests (pytest)

Task 7 - Tests specific to ML components (training, inference)

Task 8 - Model-quality and data-quality metrics

Task 9 - Testing in production and security

Appendix A - How to run; Appendix B - Deliverables

1. Assignment I recap

Assignment I (Group 14, BrakeGuard.ai v1.0) defined the problem, the requirements (with GR4ML Business, Analytics Design and Data Preparation views) and the architecture, and delivered a first runnable prototype. Assignment II takes that baseline and adds implementation quality and testing. The points that shape this report are summarised below.

1.1 Problem and ML formulation

Element	Assignment I definition
Problem	Fixed service intervals and reactive inspection miss brake systems that are degrading before a fault becomes visible.
Goal	Estimate each vehicle's probability of a brake-related fault so authorised service personnel can prioritise early inspection.
ML task	Binary classification: y = 1 if a brake-related fault occurs within the next 30 days or 1,000 km (whichever comes first).
Output	Fault probability in [0, 1] and a risk band: High >= 0.70, Medium 0.40-0.69, Low < 0.40, plus contributing factors.
Scope boundary	Decision support only: no brake-actuator or vehicle control, no automatic approval of safety-critical repairs.
Users	Service manager, service technician, fleet operator (plus data engineer, data scientist, system administrator).

1.2 Requirements carried into Assignment II

The table lists the Assignment I requirements that Assignment II implements or measures, with the status reached in this submission.

ID	Assignment I requirement	Assignment II status / evidence
AR-01	Recall >= 90 % (catch future brake faults)	Not yet met: hold-out recall 0.809 on synthetic data. Registration gate set at 0.75 (Task 8); closing the gap needs real fleet data
AR-02	Precision >= 75 %	Met: 0.769
AR-03	Calibrated probability in [0, 1]	Met: range checked in tests; ECE 0.034, Brier 0.160
AR-04 / AR-05	Report recall, precision, F1, PR-AUC per release; compare with a baseline	Met: metrics stored in every ModelBundle; tested against a random baseline (a Logistic-Regression baseline is future work)
AR-07	Temporal train/test split	Not applicable yet: synthetic data has no timestamps, so a stratified split is used
AR-08 / FR-07	Explanations for >= 90 % of predictions	Met: every prediction returns its top-3 contributing factors
FR-02 / FR-11 / DR-06	Validate data; reject impossible values; >= 95 % completeness	Met: DataValidator gate (max 5 % missing per column) and API input validation (HTTP 422)

ID	Assignment I requirement	Assignment II status / evidence
FR-06 / SR-03 / SR-09	Show timestamp and model version; log every prediction	Met: AuditLog records id, vehicle, time, probability, risk, model version, latency
SR-01	P95 prediction latency <= 500 ms	Met: 18-27 ms per prediction in the test environment
SR-04	No anonymous access to predictions	Implemented: optional X-API-Key authentication on /api/v1/*
SR-05	Fallback when the model is unavailable	Met: /ready and predictions return 503 with a clear error; automatic retraining if the artefact is missing or corrupt
SR-06	Data-quality monitoring (completeness, range, drift)	Met: data-quality report plus PSI / KS drift on live telemetry
SR-07 / SR-08	Reproducible releases; model rollback	Partly: versioned ModelBundle with metrics and library version; one-click rollback is future work
SR-10	No safety-critical control commands	Met: the system only returns predictions

1.3 Architecture decisions

- **Patterns:** Microservices (prediction, feature, explanation, audit and training services behind an API gateway) and Event-Driven Architecture (vehicle telemetry published once as `BrakeTelemetryReceived` and consumed by several services).
- **Top three quality requirements:** predictive effectiveness (recall >= 90 %, precision >= 75 %), performance (P95 <= 500 ms) and availability (>= 99.5 % monthly).
- **Planned stack:** Python, FastAPI, Kafka, a feature store, MLflow, DVC, Docker, Kubernetes, PyTest and Evidently AI.
- **Prototype delivered:** RandomForest model, in-memory feature store and event bus, FastAPI API with dashboard, Docker packaging and 4 tests. Its code is the starting point refactored in this assignment.

2. System overview

BrakeGuard AI predicts the probability that a vehicle's brake system will develop a fault within the next 30 days / 1,000 km, using six telemetry signals: rotor temperature, line pressure, pad wear, caliper vibration, brake-fluid age and odometer. A Random-Forest classifier produces a probability, which is mapped to a LOW / MEDIUM / HIGH risk band (thresholds 0.40 / 0.70) with an explanation of the main drivers. The ML component sits inside non-ML components: a REST API, a feature store, an event bus for telemetry, an audit log and drift monitoring.

Assignment I delivered a working prototype (`brakeguard_ai-2`) with module-level globals, no data validation, one happy-path test file, no logging, and a dashboard bug (the audit table read `fault_probability` while the log stored `probability`). Assignment II rebuilds it as the production-style package below. The model family and hyper-parameters are kept, so the comparison isolates software-engineering changes.

Figure 1 is the system architecture designed in Assignment I. The table after it shows where each component lives in the Assignment II code. Components that would be separate services in production run in one process here, as in the prototype; the module boundaries match the planned service boundaries.

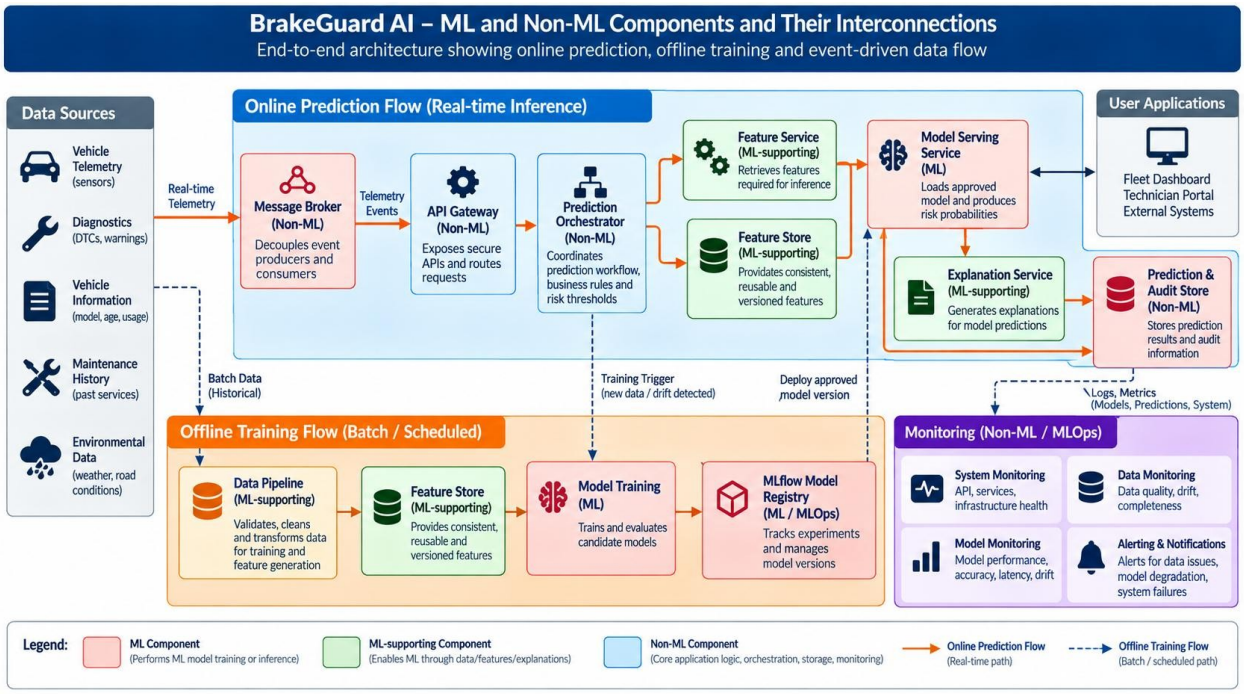


Figure 1 - Assignment I system architecture: ML and non-ML components, online prediction flow, offline training flow and monitoring (from the Group 14 Assignment I report).

Assignment I component	Type	Assignment II implementation	Status
API Gateway	Non-ML	FastAPI app factory `create_app`; X-API-Key check; request-id middleware	Implemented
Prediction Orchestrator	Non-ML	API routes + `ServiceContainer`; `RiskThresholds` maps probability to risk band	Implemented
Feature Service / Feature Store	ML-supporting	`FeatureStore` (thread-safe, in-memory); `FeatureEngineer` inside the model pipeline	Implemented (in-memory)

Assignment I component	Type	Assignment II implementation	Status
Model Serving Service	ML	`BrakeFaultPredictor` loading a registered `ModelBundle`	Implemented
Explanation Service	ML-supporting	`Prediction.top_factors`: importance x | z-score| per feature	Implemented
Prediction & Audit Store	Non-ML	`AuditLog` (bounded, thread-safe); `/api/v1/audit`	Implemented (in-memory)
Message Broker (Kafka)	Non-ML infrastructure	`EventBus` publish/subscribe with failure isolation; feature-store and drift consumers	In-process stand-in
Data Pipeline	ML-supporting	`TelemetryDataSource` sources, `DataValidator` quality gate, `FeatureEngineer`	Implemented
Model Training	ML	`ModelTrainer` + `TrainingPipeline` with data- and model-quality gates	Implemented
MLflow Model Registry	ML / MLOps	`ModelRegistry` (versioned joblib bundle with metrics, data quality and library version)	File-based stand-in
Monitoring (Evidently AI)	Non-ML / MLOps	`/api/v1/monitoring`: latency p50/p95, model quality, PSI/KS drift; structured logging	Implemented (own code)
Docker / Kubernetes	Infrastructure	Multi-stage Dockerfile with lint/test gate, docker-compose, /health and /ready probes	Docker done; K8s manifests future work

The Assignment II package is organised as follows:

Package / module	Responsibility	Key classes / functions
`brakeguard/config.py`	Immutable, environment-overridable settings	`Settings`, `RiskThresholds`, `ModelHyperParams`
`brakeguard/schemas.py`	Single source of truth for the telemetry contract	`TELEMETRY_SCHEMA`, `FeatureSpec`
`brakeguard/data/`	Data ingestion; data-quality validation	`TelemetryDataSource` (ABC), `SyntheticTelemetrySource`, `CSVTelemetrySource`, `DataValidator`, `DataQualityReport`
`brakeguard/features/`	Feature engineering; online feature store	`FeatureEngineer` (sklearn transformer), `FeatureStore`
`brakeguard/models/`	Training; versioned persistence; inference	`ModelTrainer`, `ModelRegistry`, `ModelBundle`, `BrakeFaultPredictor`, `Prediction`
`brakeguard/evaluation/`	Model-quality metrics	`evaluate_classifier`, `expected_calibration_error`
`brakeguard/monitoring/`	Audit log, pub/sub, drift	`AuditLog`, `EventBus`, `DriftDetector`, `population_stability_index`

Package / module	Responsibility	Key classes / functions
`brakeguard/pipeline.py`	Orchestration with quality gates	`TrainingPipeline`, `load_or_train`
`brakeguard/api/`	REST layer	`create_app`, routes, schemas, error handlers, API-key security, `ServiceContainer`
`scripts/`	CLIs	`train.py`, `quality_report.py`, `build_notebooks.py`

Objective 1 - Implementation and code sharing

Task 1 - OOP / functional design of the ML application

The code is organised around the ML lifecycle, with one class per responsibility. The design follows these principles:

- **Single responsibility:** ingestion, validation, feature engineering, training, persistence and inference are separate classes. Each can be tested on its own (Task 6) and replaced independently.
- **Dependency inversion:** `TrainingPipeline` depends on the abstract `TelemetryDataSource`, so synthetic data, a CSV export or a database are interchangeable. The integration test trains from a CSV without code changes.
- **Composition over globals:** the API builds a `ServiceContainer` (predictor, feature store, event bus, audit log, drift detector) per application instance through the `create_app` factory. Tests create isolated apps with no model or with an API key, without shared mutable state.
- **Functional core:** metrics (`evaluate_classifier`, `expected_calibration_error`, `population_stability_index`) are pure functions of their inputs, and settings and value objects are frozen dataclasses.
- **Training/serving consistency:** `FeatureEngineer` is an sklearn transformer placed inside the model `Pipeline`. The same fitted transformation, including imputation medians and the derived features `pressure_deviation`, `thermal_wear_index` and `fluid_age_years`, is serialised with the model and replayed at inference time.

The training workflow with its two quality gates (data quality before training, model quality before registration):

brakeguard/pipeline.py - TrainingPipeline.run

```
    def run(self, register: bool = True) -> ModelBundle:
        """Execute the pipeline and (optionally) register the model."""
        logger.info("Training pipeline started (model version %s)", self.settings.model_version)
        frame = self.source.load()
        quality = self.validator.validate_or_raise(frame, require_target=True)
        result = self.trainer.train(frame)
        self._check_quality_gates(result.test_metrics.to_dict())
        bundle = ModelBundle(
            pipeline=result.pipeline,
            version=self.settings.model_version,
            metrics=result.test_metrics.to_dict(),
            feature_names=list(ENGINEERED_FEATURES),
            reference_frame=result.reference_frame.sample(
                n=min(1000, len(result.reference_frame)), random_state=self.settings.random_seed
            ).reset_index(drop=True),
            data_quality=quality.to_dict(),
        )
        if register:
            self.registry.save(bundle)
        logger.info("Training pipeline finished")
        return bundle
```

brakeguard/features/engineering.py - FeatureEngineer.transform

```
    def transform(self, X: pd.DataFrame) -> pd.DataFrame:  # noqa: N803
        """Return a frame with raw + derived features, imputed."""
        if not hasattr(self, "medians_"):
```

```
            raise FeatureEngineeringError("FeatureEngineer must be fitted before transform")
        frame = self._check_input(X).copy()
        n_missing = int(frame.isna().sum().sum())
        if n_missing:
            logger.warning("Imputing %d missing value(s) with training medians", n_missing)
            frame = frame.fillna(value=self.medians_)
        frame["pressure_deviation"] = (frame["brake_pressure"] - self.nominal_pressure).abs()
        frame["thermal_wear_index"] = frame["brake_temperature"] * frame["pad_wear_pct"] / 100.0
        frame["fluid_age_years"] = frame["fluid_age_days"] / 365.0
        return frame[list(ENGINEERED_FEATURES)]
```

Task 2 - Research code vs production code

Component compared: **model training and inference**. The research version is `notebooks/01_research_prototype.ipynb`, written as typical exploratory cells with the same logic as Assignment I's `model_service.py`. The production version is the `brakeguard` package plus the `scripts/train.py` CLI. The notebook is kept on purpose, and its outputs show two defects the production code removes:

- Its explanation heuristic (`importance x raw value`) always ranks `odometer_km` first (score 7,674 vs 40 for the next feature) because odometer values are in tens of thousands of km. Production uses `importance x |z-score|`, and the top drivers for BG-10245 become thermal_wear_index, pad_wear_pct and vibration_rms.
- It scores physically impossible input (`pad_wear_pct = 450` gives p = 0.96). Production rejects this with HTTP 422.

Research (notebook cell)

```
# research notebook - globals, magic numbers, implicit feature order, prints
model = RandomForestClassifier(n_estimators=160, max_depth=8, min_samples_leaf=4,
random_state=42, class_weight="balanced")
model.fit(X_train, y_train)
print("recall", recall_score(y_test, pred))
joblib.dump(model, "model_research.joblib")          # relative path, no metadata
vehicle = [186.4, 72.1, 78.0, 3.2, 410, 84250]       # which number is which?
prob = model.predict_proba([vehicle])[0, 1]
risk = "HIGH" if prob >= 0.70 else "MEDIUM" if prob >= 0.40 else "LOW"
```

Production (package API)

```
# production - typed, validated, logged, versioned, tested
settings = get_settings()                                  # env-overridable, frozen
bundle = TrainingPipeline(settings=settings).run()         # validate -> train -> gate ->
register
predictor = BrakeFaultPredictor(bundle, settings.thresholds)
prediction = predictor.predict(feature_store.get("BG-10245"))   # dict keyed by schema names
prediction.risk, prediction.fault_probability, prediction.top_factors
```

Concern	Research notebook	Production package
Structure	Linear cells and global variables; results depend on execution order	Classes with single responsibilities, wired by `TrainingPipeline`

Concern	Research notebook	Production package
Configuration	Magic numbers and hard-coded paths	`Settings` dataclass + `BRAKEGUARD_*` environment variables
Data contract	Implicit NumPy column order	`TELEMETRY_SCHEMA` shared by validator, model and API
Data quality	None	`DataValidator` gate (schema, missing, range, labels, duplicates)
Model acceptance	Human reads printed metrics	Automatic gate: recall >= 0.75, PR-AUC >= 0.70
Persistence	Bare model, relative path	`ModelBundle`: version, metrics, timestamp, sklearn version, drift reference
Errors / logs	Tracebacks / print	Typed exceptions -> HTTP codes; `logging` with levels
Tests / style	None	96 pytest tests (97 % coverage); pylint 10.00/10, flake8 0

Task 3 - Error handling and logging

All modules log through the `brakeguard.*` logger hierarchy configured in `logging_config.py` (format `time | level | module | message`). Levels have a fixed meaning: **INFO** for normal lifecycle events, **WARNING** for degraded but recoverable situations, and **ERROR** for failed operations where the caller receives an error. Failures raise typed exceptions from `exceptions.py`, which `api/errors.py` maps to HTTP status codes, so the API never returns a raw stack trace.

Critical function	INFO	WARNING	ERROR / exception
`CSVTelemetrySource.load`	rows loaded	file has no rows	file missing / unparsable -> `DataIngestionError`
`DataValidator.validate_or_raise`	validation passed	duplicates; missing values below gate; issues found	data set rejected -> `DataValidationError`
`FeatureEngineer.transform`	-	missing values imputed with training medians	missing / non-numeric columns -> `FeatureEngineeringError` (422)
`ModelTrainer.train`	rows, positive rate, hold-out metrics	-	single-class target / fit failure -> `ModelTrainingError`
`TrainingPipeline` quality gate	model passed gates	-	model rejected -> `ModelTrainingError`, nothing registered
`ModelRegistry.load` / `load_or_train`	model loaded / registered	sklearn version mismatch; artefact missing -> retrain fallback	corrupt artefact -> `ModelRegistryError`
`BrakeFaultPredictor.predict*`	records scored	-	no model -> `ModelNotReadyError` (503)

Critical function	INFO	WARNING	ERROR / exception
`FeatureStore.get`	vehicle inserted / updated	unknown vehicle	`VehicleNotFoundError` (404)
`EventBus.publish`	subscriber registered	event with no successful subscriber	failing subscriber logged with traceback; others still run
`AuditLog.record` / `DriftDetector`	drift summary	HIGH-risk prediction; PSI >= 0.10 / 0.25	-
HTTP middleware / handlers	method, path, status, latency per request	422 / 404 responses	unexpected exception -> generic 500 with request id

brakeguard/models/registry.py - error handling with meaningful levels

```
    def load(self) -> ModelBundle:
        """Load a bundle; raises :class:`ModelRegistryError` on any problem."""
        if not self.exists():
            raise ModelRegistryError(f"No model artefact at {self.path}")
        try:
            bundle = joblib.load(self.path)
        except Exception as exc:  # joblib can raise many unpickling errors
            logger.error("Model artefact %s is corrupt: %s", self.path, exc)
            raise ModelRegistryError(f"Corrupt model artefact: {exc}") from exc
        if not isinstance(bundle, ModelBundle):
            logger.error("Artefact %s has unexpected type %s", self.path, type(bundle).__name__)
            raise ModelRegistryError("Artefact is not a ModelBundle")
        if bundle.sklearn_version != sklearn.__version__:
            logger.warning(
                "Model trained with scikit-learn %s, running %s - consider retraining",
                bundle.sklearn_version,
                sklearn.__version__,
            )
        logger.info("Loaded model %s (trained %s)", bundle.version, bundle.trained_at)
        return bundle
```

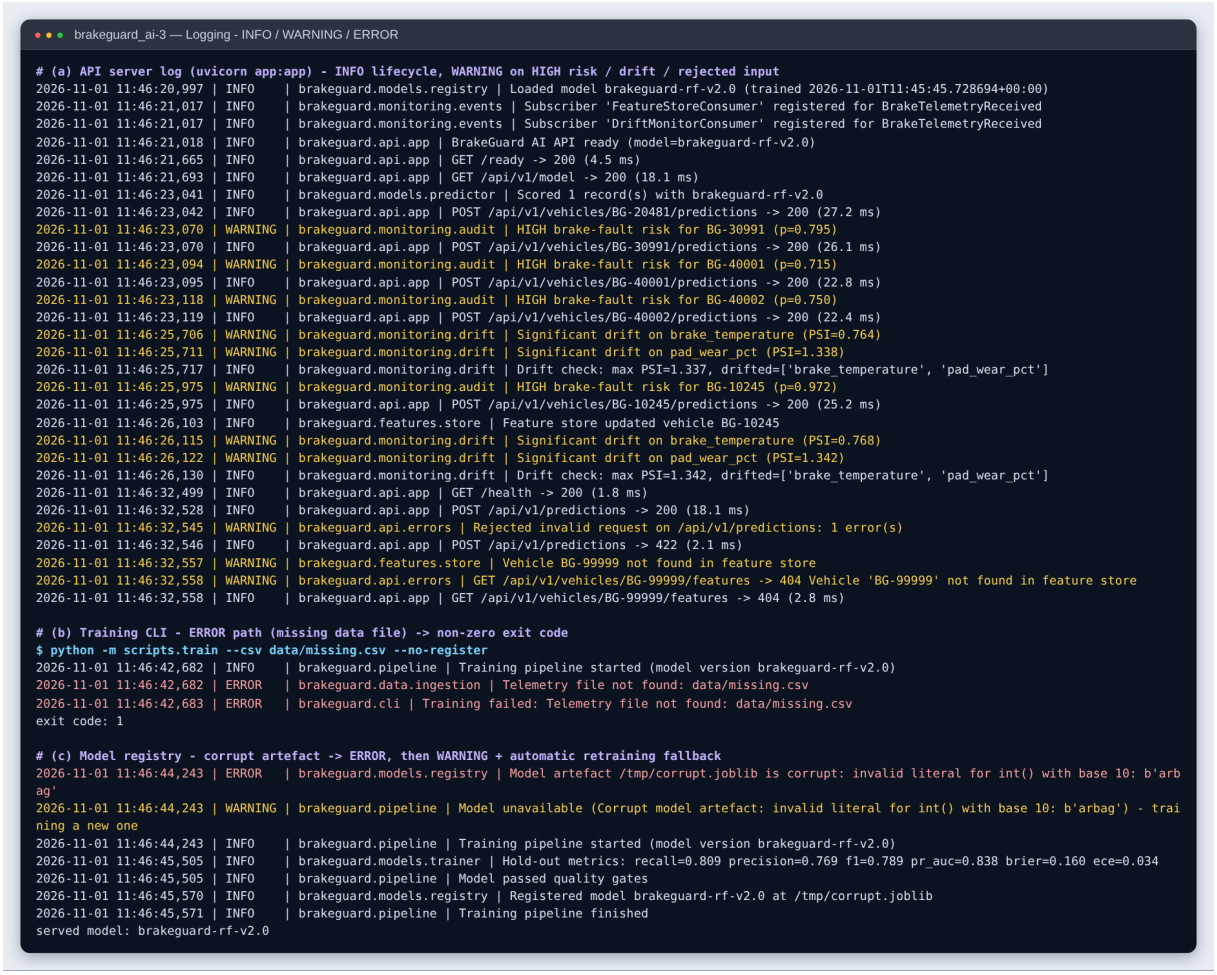


Figure 2 - Real log output: INFO lifecycle, WARNING for HIGH risk / drift / rejected input, ERROR for a missing data file (CLI exits with code 1) and a corrupt model artefact (automatic retraining fallback).

Task 4 - Code formatting and linting

Tools: **black** (formatting, line length 110), **isort** (import order, black profile), **flake8** (PEP 8 / pyflakes) and **pylint** (static analysis). Configuration is in `pyproject.toml` and `.flake8`. The same commands run as a build gate in the Dockerfile. Two "before" baselines were captured: the Assignment I code base, and the new package before black and isort were run and findings fixed.

Check	Assignment I code (before)	New package before fixes	Final package (after)
black	6 of 8 files would be reformatted	7 of 44 files would be reformatted	all 46 files unchanged
isort	3 files incorrectly sorted	clean	clean
flake8	37 issues (E231 x15, E302 x13, E501 x7, E305 x2)	12 issues (E501)	0 issues
pylint	7.25 / 10 (missing docstrings, long lines, too many args, ...)	9.83 / 10	10.00 / 10

Fixes applied to the package after the baseline: automatic reformatting (black) and import sorting (isort); renaming `VehicleId` to `VEHICLE_ID_PATH` (constant naming); removing variable shadowing (`audit` ->

`entry`); marking intentionally unused arguments; and documenting why sklearn's `fit()`-time attributes and the dataclass field counts are acceptable. Suppressions are local and commented, never global. Raw reports are in `reports/lint/`.

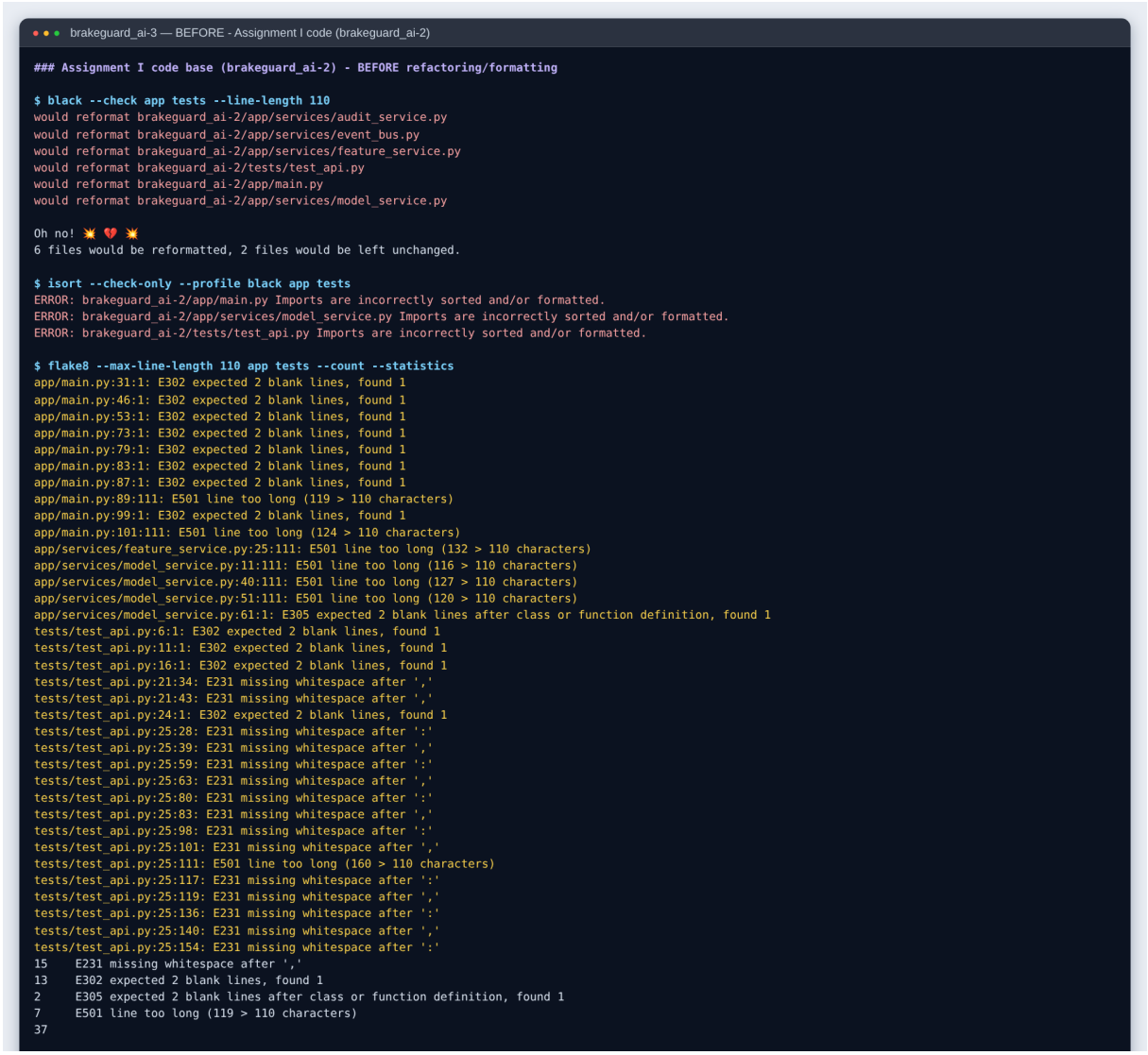
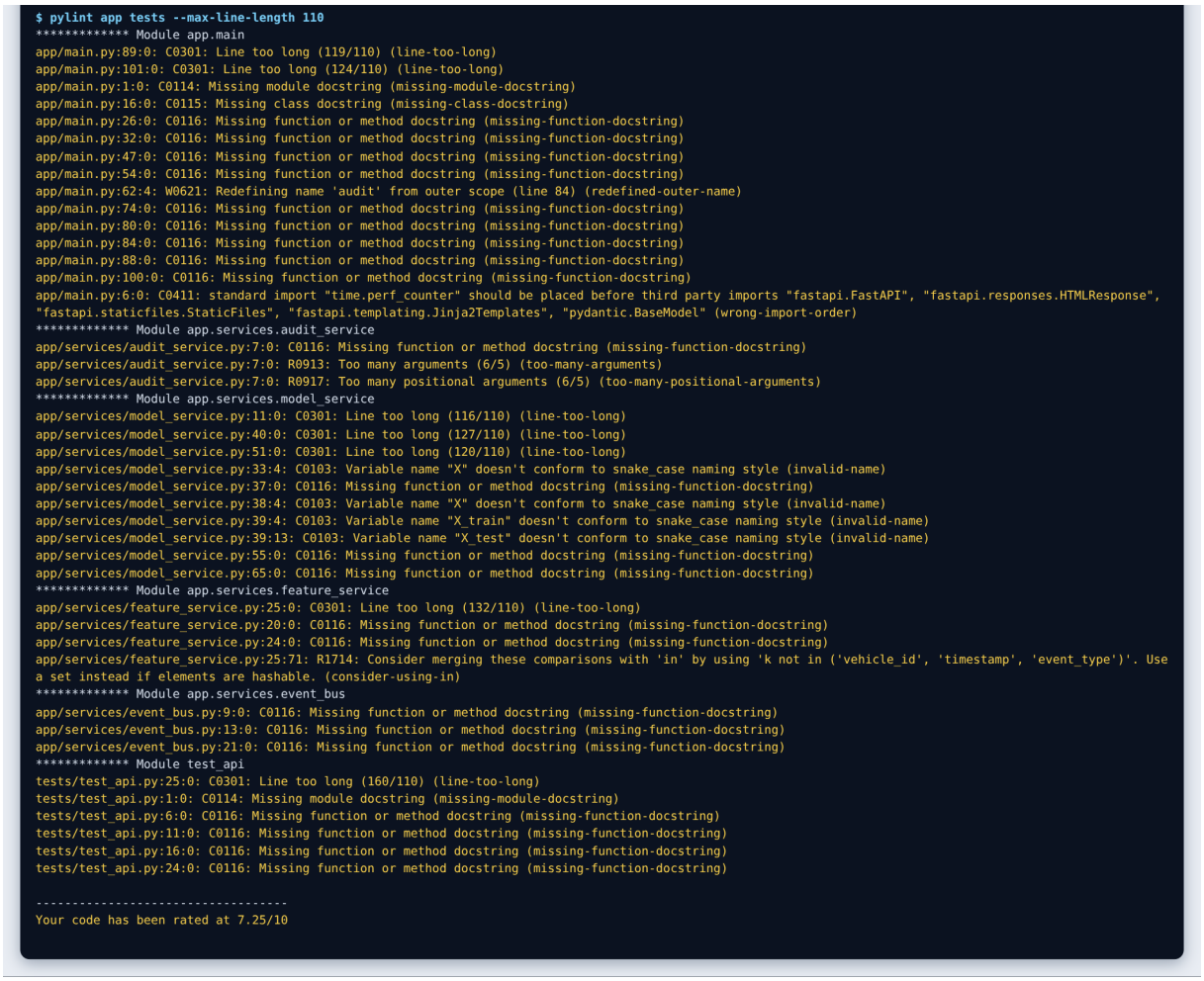


Figure 3a - BEFORE: black, isort and flake8 on the Assignment I code base.



```
$ pylint app tests --max-line-length 110
************* Module app.main
app/main.py:89:0: C0301: Line too long (119/110) (line-too-long)
app/main.py:101:0: C0301: Line too long (124/110) (line-too-long)
app/main.py:1:0: C0114: Missing module docstring (missing-module-docstring)
app/main.py:16:0: C0115: Missing class docstring (missing-class-docstring)
app/main.py:26:0: C0116: Missing function or method docstring (missing-function-docstring)
app/main.py:32:0: C0116: Missing function or method docstring (missing-function-docstring)
app/main.py:47:0: C0116: Missing function or method docstring (missing-function-docstring)
app/main.py:54:0: C0116: Missing function or method docstring (missing-function-docstring)
app/main.py:62:4: W0621: Redefining name 'audit' from outer scope (line 84) (redefined-outer-name)
app/main.py:74:0: C0116: Missing function or method docstring (missing-function-docstring)
app/main.py:80:0: C0116: Missing function or method docstring (missing-function-docstring)
app/main.py:84:0: C0116: Missing function or method docstring (missing-function-docstring)
app/main.py:88:0: C0116: Missing function or method docstring (missing-function-docstring)
app/main.py:100:0: C0116: Missing function or method docstring (missing-function-docstring)
app/main.py:6:0: C0411: standard import "time.perf_counter" should be placed before third party imports "fastapi.FastAPI", "fastapi.responses.HTMLResponse",
"fastapi.staticfiles.StaticFiles", "fastapi.templating.Jinja2Templates", "pydantic.BaseModel" (wrong-import-order)
************* Module app.services.audit_service
app/services/audit_service.py:7:0: C0116: Missing function or method docstring (missing-function-docstring)
app/services/audit_service.py:7:0: R0913: Too many arguments (6/5) (too-many-arguments)
app/services/audit_service.py:7:0: R0917: Too many positional arguments (6/5) (too-many-positional-arguments)
************* Module app.services.model_service
app/services/model_service.py:11:0: C0301: Line too long (116/110) (line-too-long)
app/services/model_service.py:40:0: C0301: Line too long (127/110) (line-too-long)
app/services/model_service.py:51:0: C0301: Line too long (120/110) (line-too-long)
app/services/model_service.py:33:4: C0103: Variable name "X" doesn't conform to snake_case naming style (invalid-name)
app/services/model_service.py:37:0: C0116: Missing function or method docstring (missing-function-docstring)
app/services/model_service.py:38:4: C0103: Variable name "X" doesn't conform to snake_case naming style (invalid-name)
app/services/model_service.py:39:4: C0103: Variable name "X_train" doesn't conform to snake_case naming style (invalid-name)
app/services/model_service.py:39:13: C0103: Variable name "X_test" doesn't conform to snake_case naming style (invalid-name)
app/services/model_service.py:55:0: C0116: Missing function or method docstring (missing-function-docstring)
app/services/model_service.py:65:0: C0116: Missing function or method docstring (missing-function-docstring)
************* Module app.services.feature_service
app/services/feature_service.py:25:0: C0301: Line too long (132/110) (line-too-long)
app/services/feature_service.py:20:0: C0116: Missing function or method docstring (missing-function-docstring)
app/services/feature_service.py:24:0: C0116: Missing function or method docstring (missing-function-docstring)
app/services/feature_service.py:25:71: R1714: Consider merging these comparisons with 'in' by using 'k not in ('vehicle_id', 'timestamp', 'event_type')'. Use
a set instead if elements are hashable. (consider-using-in)
************* Module app.services.event_bus
app/services/event_bus.py:9:0: C0116: Missing function or method docstring (missing-function-docstring)
app/services/event_bus.py:13:0: C0116: Missing function or method docstring (missing-function-docstring)
app/services/event_bus.py:21:0: C0116: Missing function or method docstring (missing-function-docstring)
************* Module test_api
tests/test_api.py:25:0: C0301: Line too long (160/110) (line-too-long)
tests/test_api.py:1:0: C0114: Missing module docstring (missing-module-docstring)
tests/test_api.py:6:0: C0116: Missing function or method docstring (missing-function-docstring)
tests/test_api.py:11:0: C0116: Missing function or method docstring (missing-function-docstring)
tests/test_api.py:16:0: C0116: Missing function or method docstring (missing-function-docstring)
tests/test_api.py:24:0: C0116: Missing function or method docstring (missing-function-docstring)

---------------------------------
Your code has been rated at 7.25/10
```

Figure 3b - BEFORE: pylint on the Assignment I code base (7.25/10).



Figure 4 - BEFORE: the new package prior to black/isort and lint fixes (9.83/10, 12 flake8 issues).

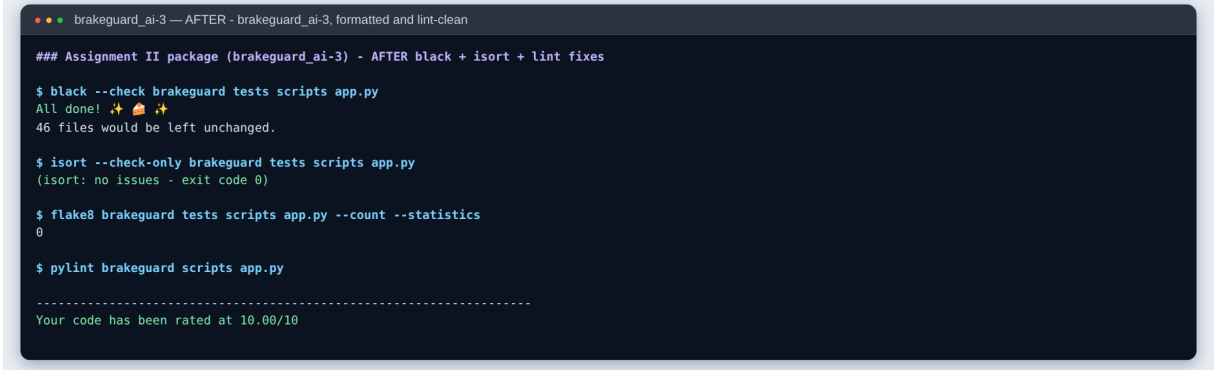


Figure 5 - AFTER: black and isort clean, flake8 0 issues, pylint 10.00/10.

Task 5 - REST API (FastAPI)

The API is created by `brakeguard.api.create_app()` and served by `uvicorn app:app`. It follows these design practices:

- **Resource-oriented, versioned URLs** under `/api/v1`, with nouns and HTTP verbs (`POST /predictions`, `GET /vehicles/{id}/features`). Operational probes (`/health` liveness, `/ready` readiness) are kept outside the versioned prefix.
- **Explicit request/response schemas** (Pydantic) published in OpenAPI/Swagger. Field ranges are derived from `TELEMETRY_SCHEMA`, unknown fields are forbidden, and batch size is limited to 1-500.
- **Meaningful status codes:** 200 OK, 202 Accepted (telemetry is processed asynchronously in production), 401 Unauthorized, 404 Not Found, 422 Unprocessable Content, 503 Service Unavailable (no model). All are documented per endpoint.
- **Uniform error envelope** `{"error": {"code", "message", "request_id", "details"}}` for every non-2xx response.
- **Traceability:** every response carries `X-Request-ID` and `X-Process-Time-ms`, and every prediction gets a `prediction_id` stored in the audit log with the model version.
- **Security:** optional `X-API-Key` authentication on `/api/v1/*` (Task 9).

Method	Endpoint	Purpose	Codes
GET	/health, /ready	Liveness / readiness probes	200 / 200, 503
GET	/api/v1/model	Model version, training time, metrics, thresholds	200, 401, 503
POST	/api/v1/predictions	Score telemetry in the request body	200, 401, 422, 503
POST	/api/v1/predictions/batch	Score up to 500 records (vectorised)	200, 401, 422, 503
GET	/api/v1/vehicles	List vehicles in the feature store	200, 401
GET	/api/v1/vehicles/{id}/features	Latest features of a vehicle	200, 401, 404, 422
POST	/api/v1/vehicles/{id}/predictions	Score a vehicle from the feature store	200, 401, 404, 503
POST	/api/v1/telemetry	Publish a BrakeTelemetryReceived event	202, 401, 422
GET	/api/v1/events, /api/v1/audit	Recent events / audited predictions	200, 401
GET	/api/v1/monitoring	Latency p50/p95, model quality, data drift	200, 401

brakeguard/api/routes.py - prediction endpoint

```
@api_router.post("/predictions", response_model=PredictionResponse, responses=ERRORS,
tags=["predictions"])
def predict(body: PredictionRequest, c: ServiceContainer = Depends(get_container)) ->
PredictionResponse:
    """Score telemetry supplied in the request body."""
    start = perf_counter()
    pred = c.predictor.predict(body.telemetry.model_dump())
    return _to_response(c, body.vehicle_id, pred, (perf_counter() - start) * 1000)
```

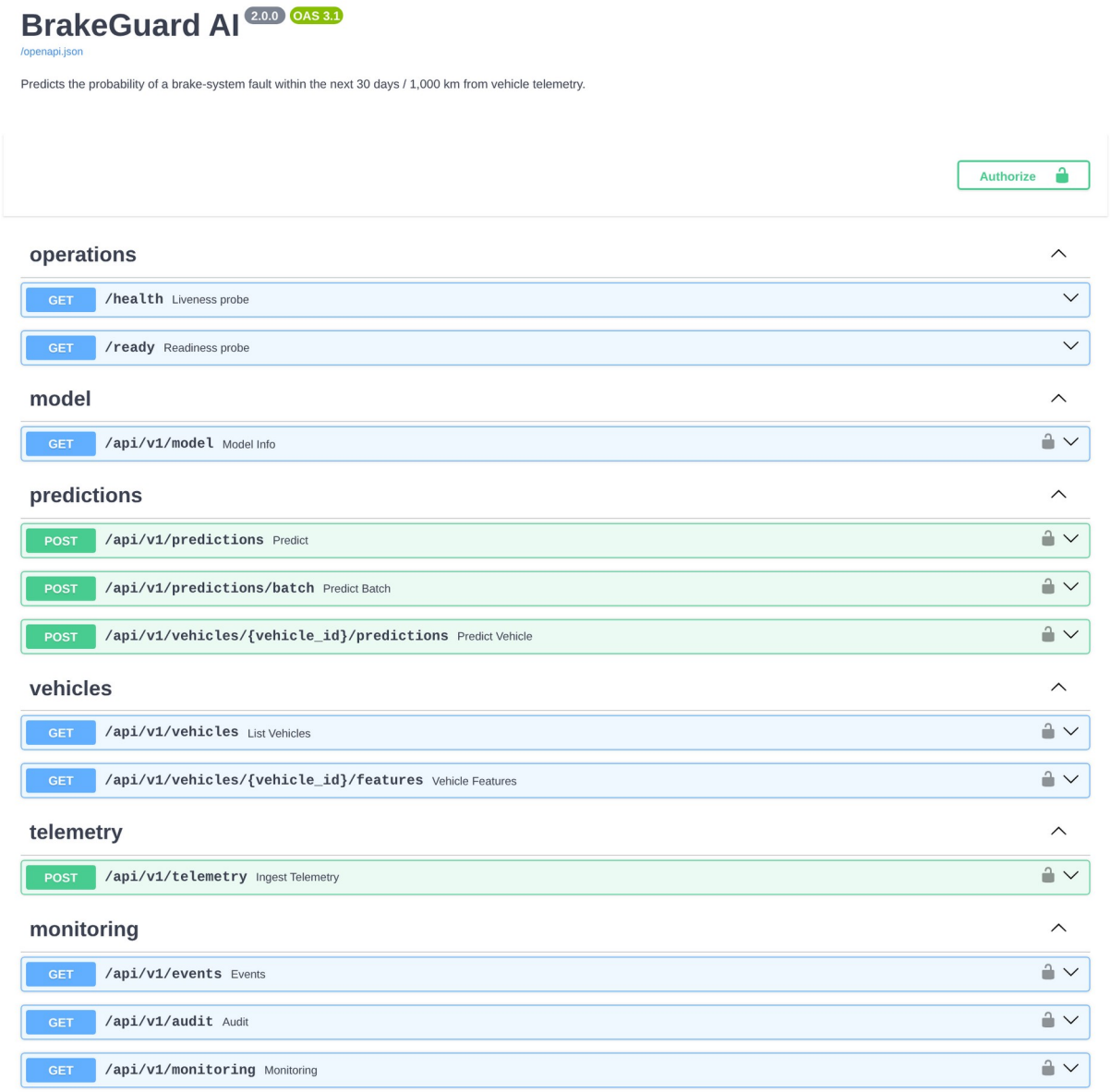


Figure 6 - Swagger UI (/docs): endpoints grouped by tag; lock icons mark API-key-protected routes.

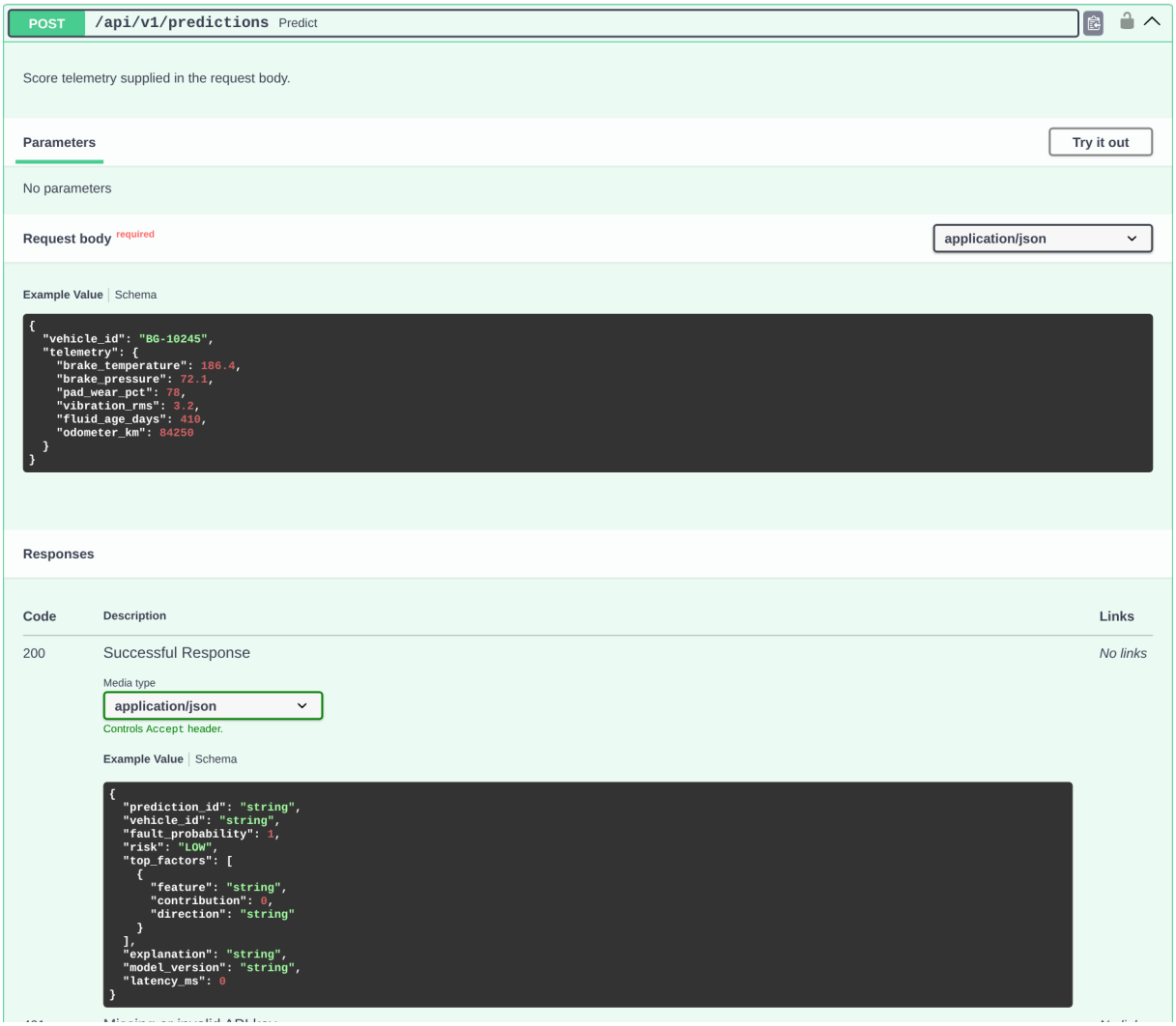


Figure 7 - Request schema with example, and documented 200/401/422/503 responses for POST /api/v1/predictions.

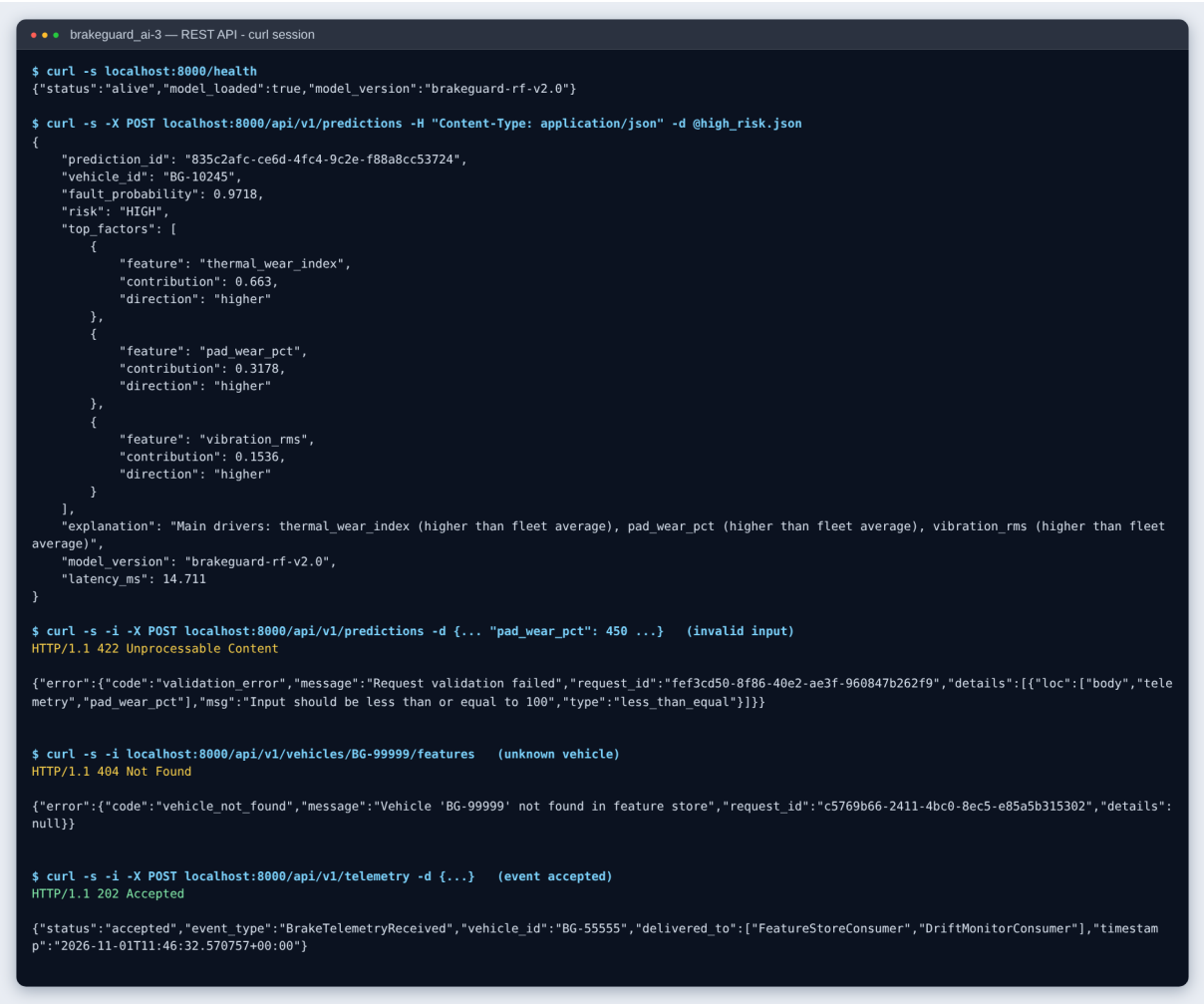


Figure 8 - curl session: 200 prediction with explanation, 422 for pad wear of 450 %, 404 for an unknown vehicle, 202 for a telemetry event.

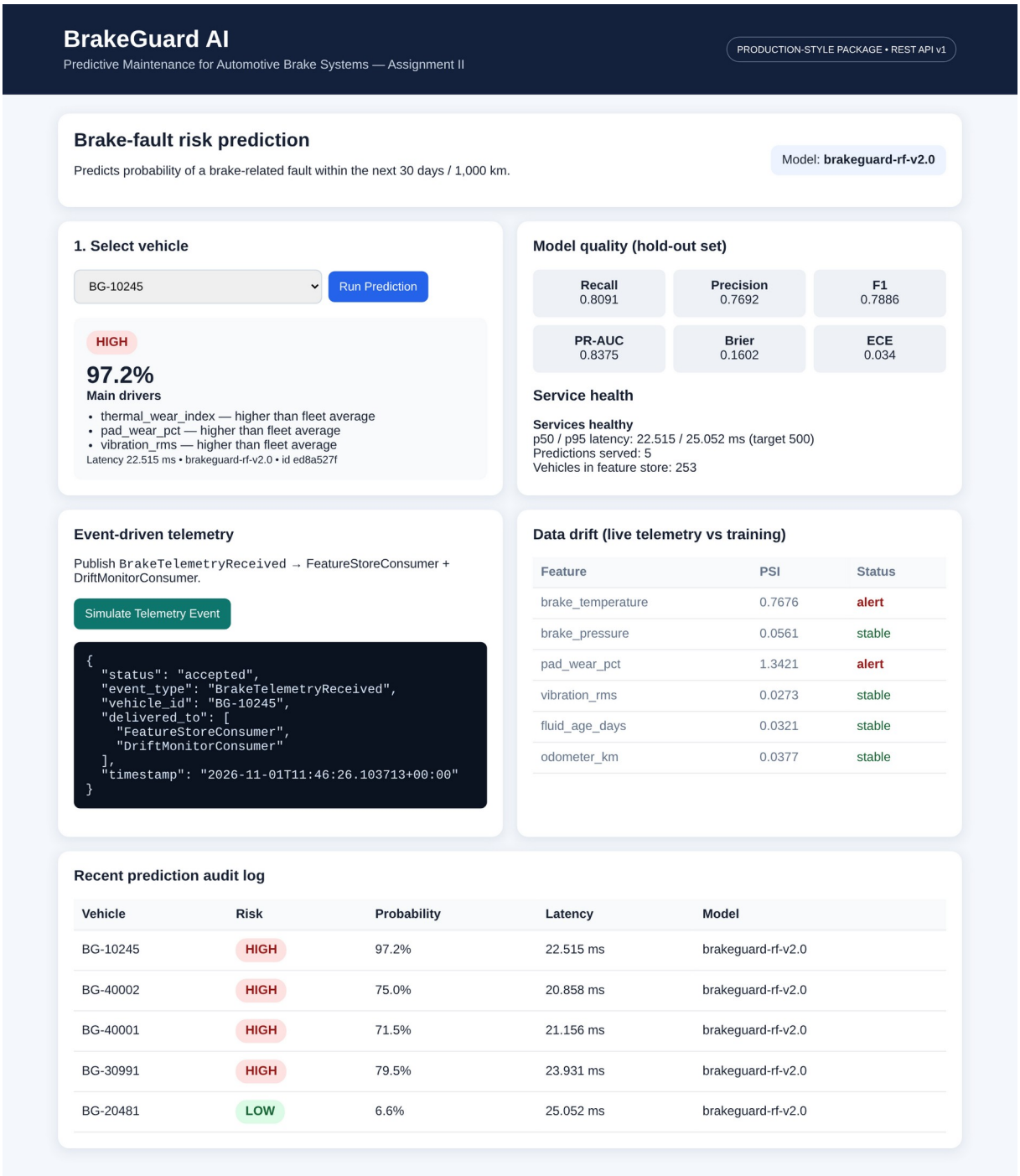


Figure 9 - Dashboard served by the API: prediction with drivers, hold-out model-quality metrics, latency, event-driven telemetry, live drift (a simulated heat-wave fleet drifts on brake temperature and pad wear only) and the audit log.

Objective 2 - Quality assurance

Task 6 - Types of tests (pytest)

The suite has **96 tests in four categories**. All pass in about 12 s, with **97 % line coverage** of the `brakeguard` package. Session-scoped fixtures in `tests/conftest.py` train the production model once. Each API test gets a fresh app instance, so tests never share state.

Type	Location	#	What is verified
Unit tests	tests/unit/	26	Risk-threshold boundaries; FeatureEngineer (derived values, median imputation + WARNING log, missing/non-numeric columns); FeatureStore copies / unknown ids; EventBus isolation of failing subscribers; bounded AuditLog; ECE / metric correctness on hand-computed cases; registry round-trip, missing and corrupt artefacts
Data-validation tests	tests/ data_quality/	19	Schema (missing columns, target), completeness gate, physical-range violations, non-numeric types, invalid labels, duplicates, empty data, CSV ingestion errors, PSI behaviour, DriftDetector on stable vs shifted fleets
ML tests	tests/ml/	24	Training behaviour (8) and inference behaviour (16); see Task 7
Integration tests	tests/integration/	27	HTTP -> services -> model end to end: every endpoint and status code, input-validation and auth rules, event-driven feature-store update, drift monitoring, CSV -> train -> register -> load, bad data blocks registration, retrain fallback, CLI exit codes

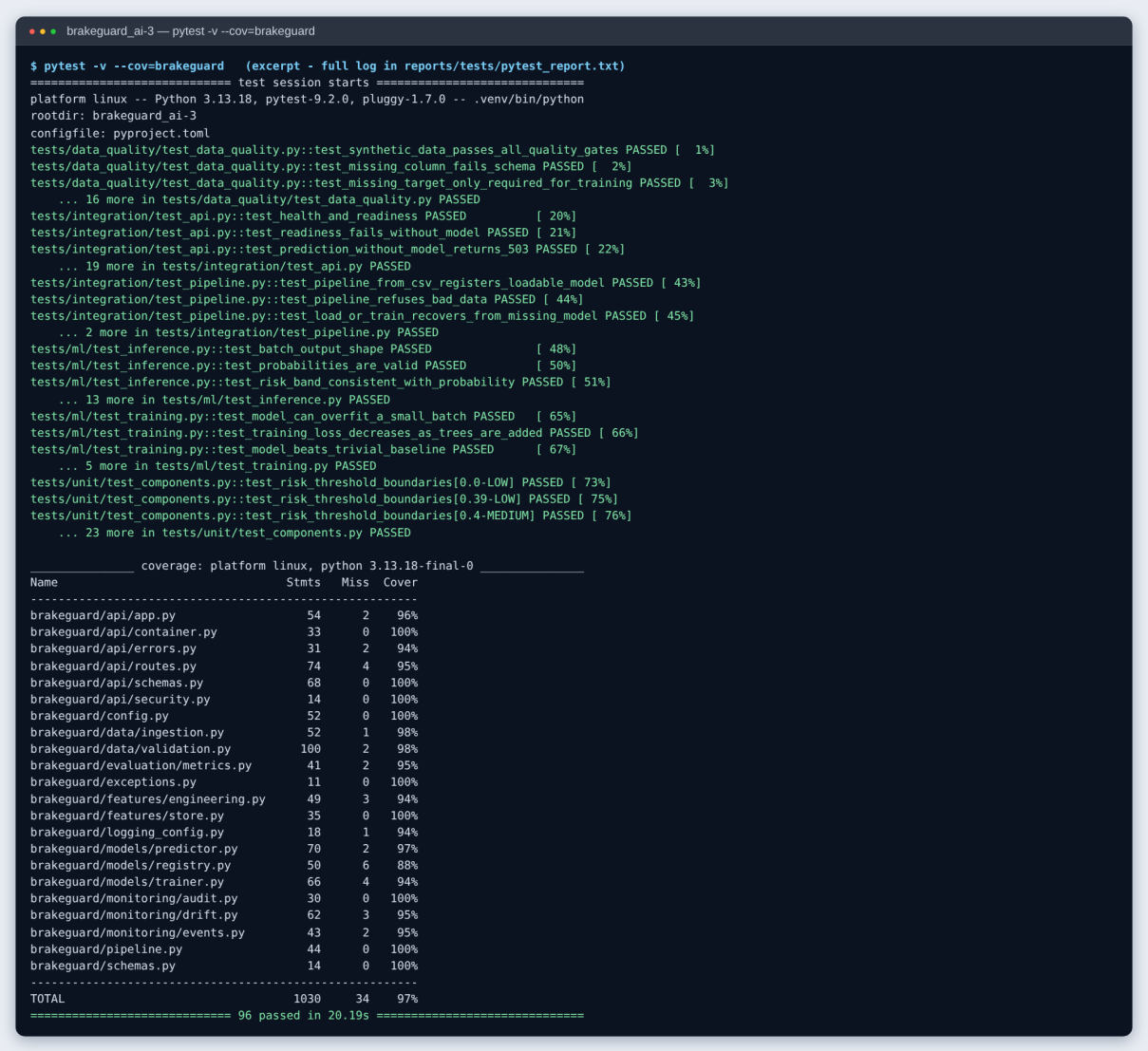


Figure 10 - pytest run (excerpt; full verbose log in reports/tests/pytest_report.txt) with the coverage table.

Task 7 - Tests specific to ML components

7a. Testing model training (tests/ml/test_training.py)

Test	Rationale
`test_model_can_overfit_a_small_batch`	A high-capacity forest must memorise 32 samples (100 % training accuracy). Failure means misaligned labels or dropped features.
`test_training_loss_decreases_as_trees_are_added`	Random forests have no epochs, so the analogue is training log-loss as trees are added (warm_start). It falls from 0.6842 (1 tree) to 0.344 (160 trees) and must never rise by more than 0.01 (Figure 11).
`test_model_beats_trivial_baseline`	PR-AUC must exceed the positive rate (a random classifier's PR-AUC) by 0.2; ROC-AUC > 0.75.
`test_model_meets_quality_targets`	Recall >= 0.75, ECE <= 0.10, Brier <= 0.20.

Test	Rationale
`test_training_is_reproducible`	Same data and seed give identical metrics.
`test_no_severe_overfitting`	Train-test ROC-AUC gap < 0.15.
`test_single_class_target_is_rejected`	Degenerate data raises `ModelTrainingError`.
`test_quality_gate_rejects_model_trained_on_noise`	With shuffled labels the pipeline refuses to register the model and logs an ERROR.

tests/ml/test_training.py

```
def test_model_can_overfit_a_small_batch():
    """A model with enough capacity must memorise 32 samples.

    If it cannot, the pipeline is broken (e.g. labels misaligned with
    features or features silently dropped) - the classic "overfit one
    batch" sanity check.
    """
    batch = SyntheticTelemetrySource(n_samples=32, seed=3).load()
    trainer = ModelTrainer()
    X, y = trainer.split_xy(batch)
    pipeline = trainer.build_pipeline(n_estimators=50, max_depth=None, min_samples_leaf=1,
class_weight=None)
    pipeline.fit(X, y)
    assert (pipeline.predict(X) == y).mean() == 1.0


def test_training_loss_decreases_as_trees_are_added():
    frame = SyntheticTelemetrySource(n_samples=600, seed=5).load()
    trainer = ModelTrainer()
    X, y = trainer.split_xy(frame)
    losses = trainer.loss_curve(X, y, steps=(1, 5, 10, 25, 50, 100))
    assert losses[-1] < losses[0] * 0.5  # big drop from a single tree to the ensemble
    # once converged the curve may plateau, but it must never climb noticeably
    assert all(later <= earlier + 0.01 for earlier, later in zip(losses, losses[1:]))
```

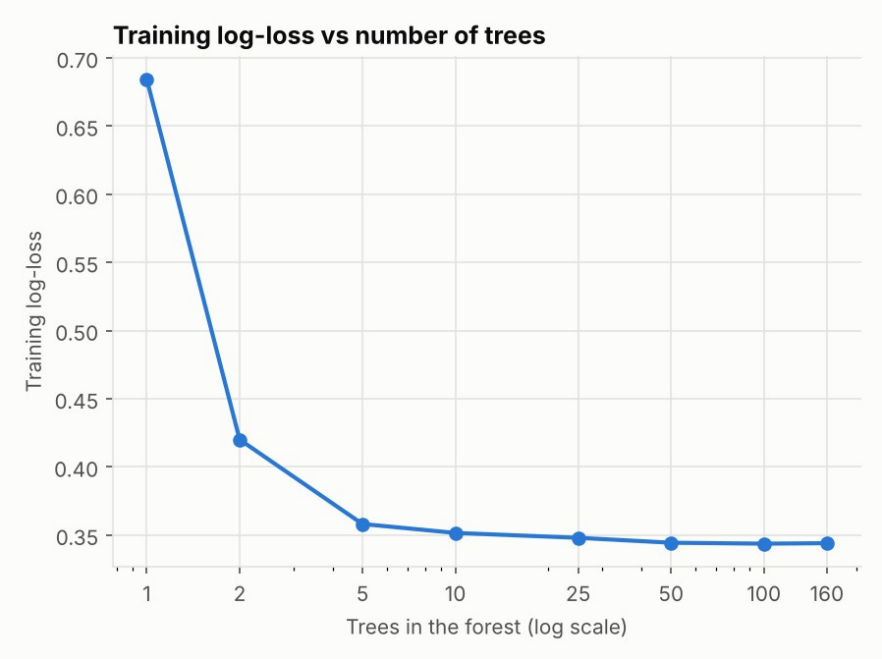


Figure 11 - Training log-loss as trees are added (basis of the 'loss decreases' test).

7b. Testing model inference (tests/ml/test_inference.py)

These tests follow the CheckList approach: shape and range checks, minimum-functionality cases, invariance tests (changes that must not affect the output) and directional-expectation tests (changes that must move the output in a known direction).

Category	Tests
Output shape / range	Batch output shape = (n,); all probabilities in [0, 1] and not NaN; risk band consistent with thresholds; explanation has top-k known features; deterministic; batch == single predictions
Minimum functionality	Obviously failing brakes (230 degC, 92 % wear, 5.5 g vibration) -> HIGH; new brakes -> LOW
Invariance	Key order of the input; extra metadata (vehicle_id, driver name); 0.1 % sensor noise changes probabilities by < 0.01 on average
Directional	Higher pad wear, vibration, temperature or fluid age raises mean fleet probability (> +0.02); line pressure 25 bar or 105 bar both score above nominal 65 bar

tests/ml/test_inference.py - directional expectation tests

```
# -- directional expectations -----------------------------------------------------
@pytest.mark.parametrize(
    ("feature", "low", "high"),
    [
        ("pad_wear_pct", 15.0, 90.0),
        ("vibration_rms", 0.5, 5.0),
        ("brake_temperature", 100.0, 240.0),
        ("fluid_age_days", 30.0, 800.0),
    ],
)
def test_worse_condition_increases_risk(predictor, fleet, feature, low, high):
```

```
    """Averaged over the fleet, a worse value must not lower the fault probability."""
    better = predictor.predict_proba([{**r, feature: low} for r in fleet])
    worse = predictor.predict_proba([{**r, feature: high} for r in fleet])
    assert worse.mean() > better.mean() + 0.02


def test_pressure_deviation_in_either_direction_increases_risk(predictor, fleet):
    nominal = predictor.predict_proba([{**r, "brake_pressure": 65.0} for r in fleet]).mean()
    too_low = predictor.predict_proba([{**r, "brake_pressure": 25.0} for r in fleet]).mean()
    too_high = predictor.predict_proba([{**r, "brake_pressure": 105.0} for r in fleet]).mean()
    assert too_low > nominal and too_high > nominal
```

Task 8 - Model-quality and data-quality metrics

All values below were measured by `python -m scripts.quality_report` (output in `reports/metrics/quality_report.json`) on a 600-row stratified hold-out set of the 3,000-row synthetic fleet data. The same model metrics are stored in the model bundle and served at `/api/v1/model` and on the dashboard. They were produced with scikit-learn 1.6.1 and match the Colab execution of `notebooks/Group14.ipynb`. Random-forest internals changed in scikit-learn 1.9, where the same code gives slightly different values (e.g. recall 0.816 instead of 0.809), so the version is recorded with every model in `ModelBundle.sklearn_version`.

8a. Model quality

Metric	Why it matters for BrakeGuard	Hold-out	Train	Target / gate
Recall	Share of real brake faults caught; a missed fault is the costly error	0.809	0.867	>= 0.75 (registration gate)
Precision	Share of alerts that are real; controls workshop false alarms	0.769	0.887	monitored
F1	Balance of recall and precision	0.789	0.876	monitored
PR-AUC	Threshold-free ranking quality on the positive class	0.838	0.958	>= 0.70 (registration gate)
ROC-AUC	Threshold-free discrimination	0.846	0.951	> 0.75 (test)
Brier score	Calibration + sharpness of probabilities (lower is better)	0.160	0.100	<= 0.20 (test)
ECE (10 bins)	Does a 70 % shown to a fleet manager mean 70 %?	0.034	0.097	<= 0.10 (test)

Recall is 0.809 at threshold 0.5. Across other synthetic seeds hold-out recall varied between about 0.70 and 0.82, which is why recall is a gate rather than a single trusted number, and why it is re-checked online (Task 9). Calibration is good (ECE 0.034), so the probability shown on the dashboard can be read at face value.

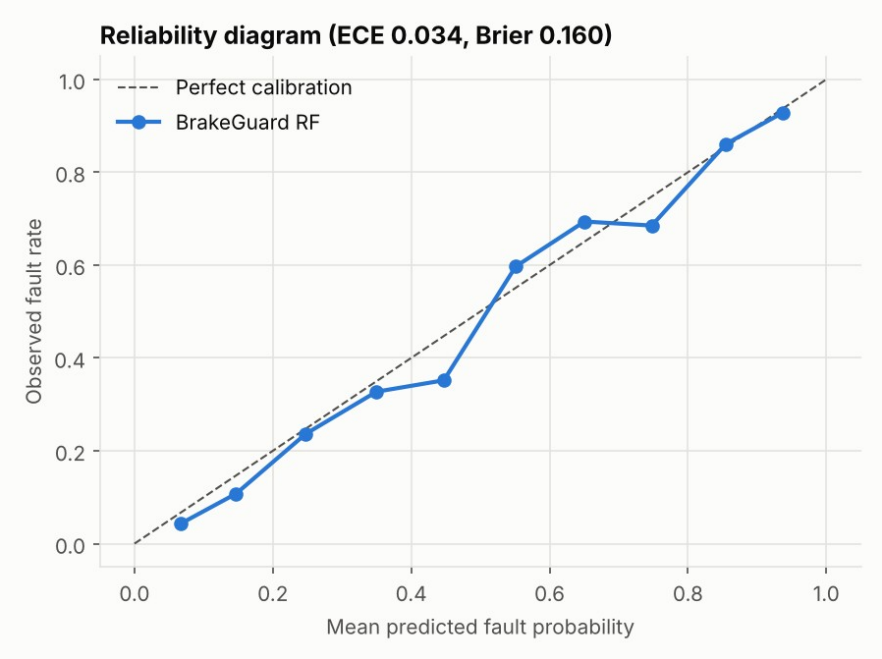


Figure 12 - Reliability diagram: predicted probabilities track observed fault rates.

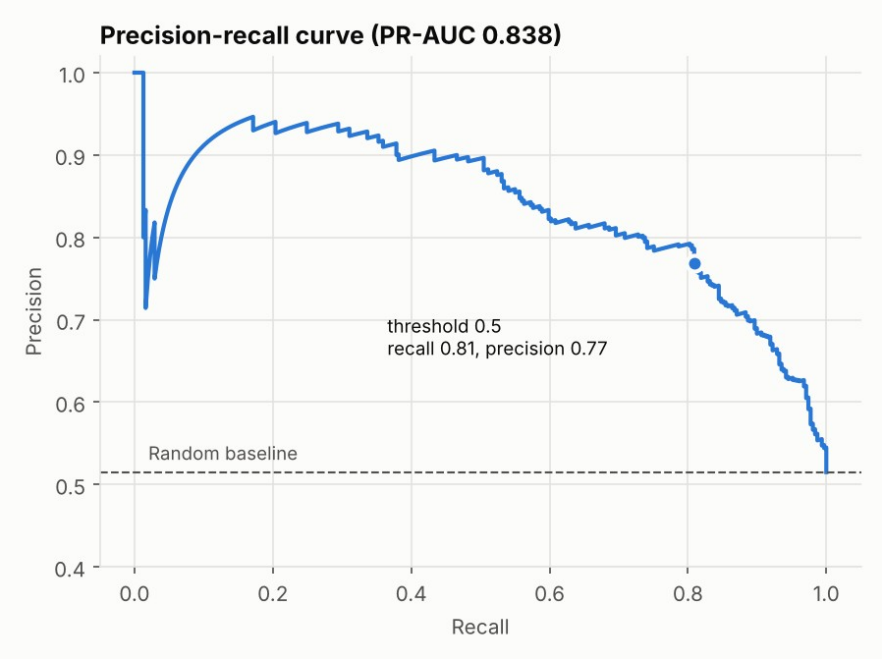


Figure 13 - Precision-recall curve with the operating point at threshold 0.5.

8b. Data quality

`DataValidator` measures data-quality metrics for every batch, and `DriftDetector` compares live telemetry with the training reference. They were run on a clean batch, a deliberately corrupted batch (sensor dropouts, a fraction-vs-percent unit bug and duplicated rows), and a stable and a shifted fleet:

Metric	Definition	Clean batch	Corrupted batch	Gate
Schema validity	All required columns present and numeric	true	true	must be true

Metric	Definition	Clean batch	Corrupted batch	Gate
Missing-value ratio	Share of null cells per column	0.00 %	7.77 % (brake_temperature)	<= 5 % per column
Out-of-range ratio	Values outside the physical range in TELEMETRY_SCHEMA	0.00 %	5.05 % (pad_wear_pct)	<= 1 % per column
Duplicate-row ratio	Exact duplicate records	0.00 %	2.91 %	warning
Gate result		PASSED	REJECTED	

Drift metric	Stable fleet (max over features)	Shifted fleet	Thresholds
PSI (10 reference-quantile bins)	0.021	brake_temperature 1.29, pad_wear_pct 1.45; others <= 0.018	warning 0.10, alert 0.25
Kolmogorov-Smirnov p-value	min 0.139	brake_temperature < 1e-6	< 0.01 confirms shift
Drifted features	none	brake_temperature, pad_wear_pct	

While building the live drift monitor, PSI over 10 bins on a 30-40-sample window flagged every feature as drifting, because the estimate is dominated by noise. The minimum window was therefore made a setting (`BRAKEGUARD_DRIFT_MIN_SAMPLES`, default 200), and an integration test checks that an in-distribution fleet of 250 vehicles raises no drift alert.

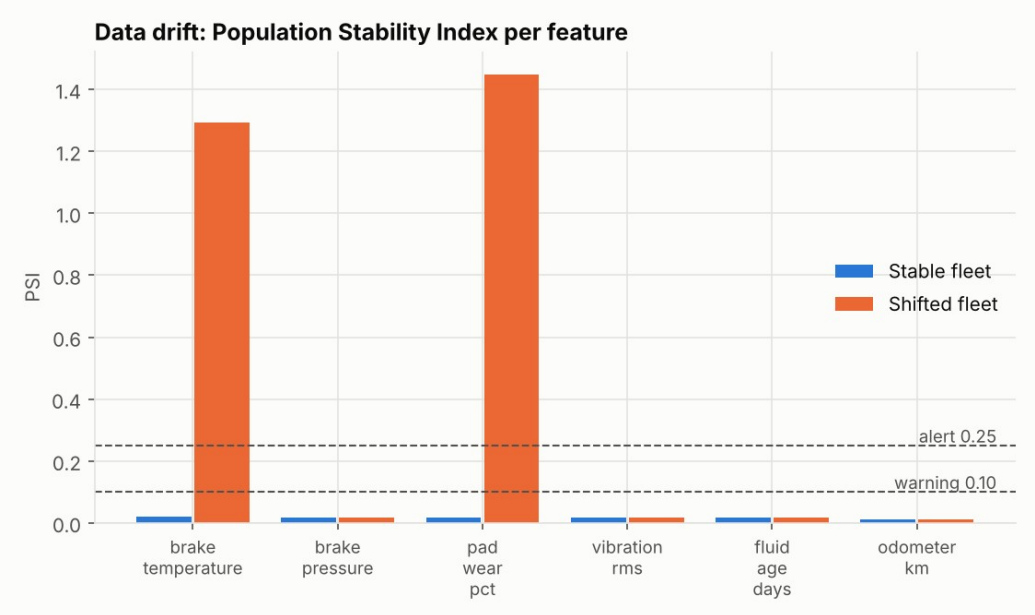


Figure 14 - PSI per feature: the shifted (heat-wave) fleet drifts on temperature and pad wear only.

Task 9 - Testing in production and security

Testing / experimentation in production: shadow -> canary -> A/B

Offline metrics come from synthetic data, and brake faults are only confirmed at the workshop weeks later. A new model version (the challenger) is therefore rolled out against the current champion in three stages:

Stage	Traffic	Exit criteria
1. Shadow (2-4 weeks)	100 % of requests mirrored to the challenger; only the champion's answer is returned and the challenger's is logged	No errors; p95 latency < 500 ms; >= 90 % agreement on HIGH/LOW; challenger recall >= champion on vehicles whose outcome is already known
2. Canary (1-2 weeks)	5 % -> 25 % -> 50 % of vehicles, sticky by vehicle_id hash	Error rate and latency not worse; HIGH-alert rate within +/-20 % of champion; automatic rollback on breach
3. A/B, then full rollout	50/50, then 100 %	After the 30-day label horizon: recall not significantly lower (one-sided test, alpha 0.05), false-alarm rate lower or equal; old version kept in the registry for one-click rollback

Shadow mode comes first because a missed brake fault is a safety incident, so a challenger must not influence a real maintenance decision before its live behaviour is known. The code base already has the needed pieces: `ModelRegistry` versions artefacts, `AuditLog` records `prediction_id` and `model_version` for joining with workshop outcomes, and `/api/v1/monitoring` reports the latency and drift guard-rail metrics that pause or roll back a rollout.

Security consideration: input validation and model access control

Threat. Crafted or faulty telemetry could suppress a real HIGH alert (an evasion attack), poison the feature store and drift statistics, or be used to query the model at high volume to reverse-engineer it. The Assignment I API and the research notebook accept any value; the notebook scores 450 % pad wear as a valid reading.

- **Schema-driven input validation:** physical ranges per field, `extra="forbid"`, a `vehicle_id` pattern and a batch cap of 500. Violations return 422 and are logged as WARNING. Covered by 6 integration tests.
- **Model access control:** when `BRAKEGUARD_API_KEY` is set, `/api/v1/*` requires an `X-API-Key` header, compared in constant time with `hmac.compare_digest`. The key comes only from the environment. Probes stay public for orchestration.
- **Safe failure:** generic 500 responses without stack traces; details are logged under the `request_id`.
- **Bounded state** (audit, event and drift windows) against memory exhaustion, and drift alerts as a detection control for coordinated data injection.
- **Hardened deployment:** multi-stage Docker build with lint and tests as a gate, non-root user, read-only filesystem.

Next steps for production: mTLS from vehicle gateways, per-client rate limiting at the gateway, and signed model artefacts. Unpickling an untrusted joblib file executes code, so only a file whose hash matches the registry entry should be loaded. Full write-up: `docs/production_testing_and_security.md`.

Appendix A - How to run

```
cd BrakeGuard_AI_Assignment_Application/brakeguard_ai-3
python -m venv .venv && source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements-dev.txt
python -m scripts.train --metrics-out reports/metrics/training_metrics.json
uvicorn app:app --reload                                  # dashboard http://127.0.0.1:8000,
Swagger /docs
pytest -v --cov=brakeguard                                # 96 tests
black --check brakeguard tests scripts app.py && isort --check-only brakeguard tests scripts
app.py
flake8 brakeguard tests scripts app.py && pylint brakeguard scripts app.py
python -m scripts.quality_report                          # metrics + figures
docker compose up --build                                 # lint + tests run inside the image
build
```

Appendix B - Deliverables in the repository

Path	Content
brakeguard/	Production package (about 2,000 lines across 29 modules)
tests/	96 pytest tests in unit / data_quality / ml / integration
notebooks/Group14.ipynb	Implementation notebook with group details, executed end to end in Google Colab
notebooks/ 01_research_prototype.ipynb	Research-code version used for the Task 2 comparison
docs/	research_vs_production.md, production_testing_and_security.md
reports/lint/	Lint reports: before (Assignment I), before (new package), after
reports/tests/	pytest verbose log + coverage, summary
reports/metrics/	quality_report.json, training_metrics.json, figures
reports/api/	curl transcript, logging evidence
reports/screenshots/	Dashboard, Swagger, lint, tests, API, logging screenshots